In [2]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, classification_report
from sklearn.decomposition import PCA
from matplotlib.colors import ListedColormap
print("done")
```

```
done
```

In [3]:
```
url = "https://archive.ics.uci.edu/ml/machine-learning-databases/00222/ba

import zipfile
import io
import urllib.request

response = urllib.request.urlopen(url)
data = response.read()

with zipfile.ZipFile(io.BytesIO(data)) as z:
    df = pd.read_csv(z.open("bank-full.csv"), sep=";")

df.head()
```

Out[3]:

	age	job	marital	education	default	balance	housing	loan	contact
0	58	management	married	tertiary	no	2143	yes	no	unknown
1	44	technician	single	secondary	no	29	yes	no	unknown
2	33	entrepreneur	married	secondary	no	2	yes	yes	unknown
3	47	blue-collar	married	unknown	no	1506	yes	no	unknown
4	33	unknown	single	unknown	no	1	no	no	unknown

In [4]:
```
# Target variable: term-deposit subscription
X = df.drop("y", axis=1)
y = df["y"].map({"no": 0, "yes": 1})

# Encode categorical features
X = pd.get_dummies(X, drop_first=True)


X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)


scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

print("Training samples:", X_train.shape[0])
```

```
print("Testing samples:", X_test.shape[0])
print("Number of encoded features:", X_train.shape[1])
```

```
Training samples: 36168
Testing samples: 9043
Number of encoded features: 42
```

In [5]:
```
knn = KNeighborsClassifier(n_neighbors=3)


knn.fit(X_train, y_train)


y_pred = knn.predict(X_test)


print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=["No", "Yes"]))
```

```
Accuracy: 0.8884219838549154

Classification Report:
              precision    recall  f1-score   support

          No       0.92      0.96      0.94      7952
         Yes       0.56      0.37      0.44      1091

    accuracy                           0.89      9043
   macro avg       0.74      0.66      0.69      9043
weighted avg       0.87      0.89      0.88      9043
```

In [6]:
```
sample = X_test[0].reshape(1, -1)
prediction = knn.predict(sample)

print("Predicted class:", "Yes" if prediction[0] == 1 else "No")
```

```
Predicted class: No
```

Visualization

In [7]:
```
# Reduce the data to two dimensions only for visualization
pca = PCA(n_components=2, random_state=42)

X_train_2d = pca.fit_transform(X_train)

knn_2d = KNeighborsClassifier(n_neighbors=3)
knn_2d.fit(X_train_2d, y_train)

# Create a mesh for the decision regions
x_min, x_max = X_train_2d[:, 0].min() - 1, X_train_2d[:, 0].max() + 1
y_min, y_max = X_train_2d[:, 1].min() - 1, X_train_2d[:, 1].max() + 1

xx, yy = np.meshgrid(
    np.arange(x_min, x_max, 0.1),
    np.arange(y_min, y_max, 0.1)
)

Z = knn_2d.predict(np.c_[xx.ravel(), yy.ravel()])
```

```
Z = Z.reshape(xx.shape)

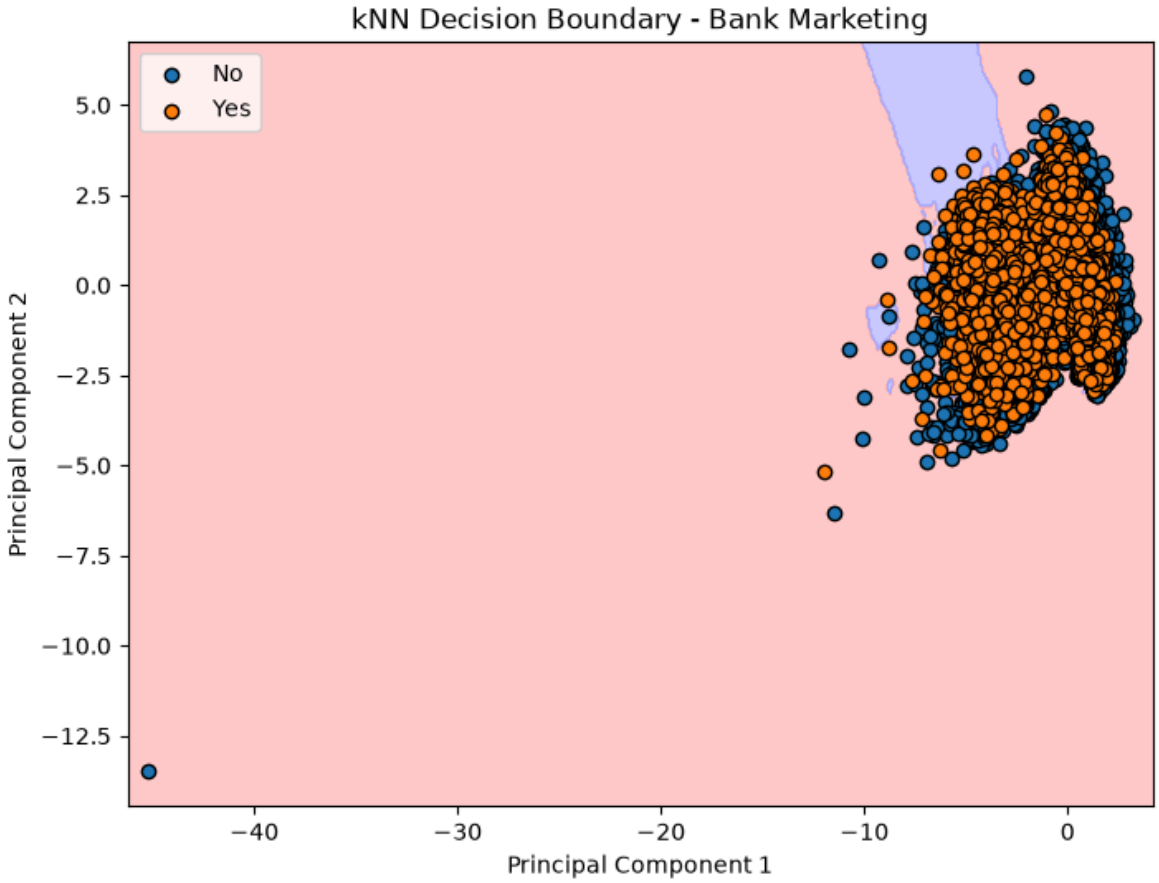
cmap_light = ListedColormap(["#FFAAAA", "#AAAAFF"])

plt.figure(figsize=(8, 6))
plt.contourf(xx, yy, Z, cmap=cmap_light, alpha=0.6)

plt.scatter(
    X_train_2d[y_train == 0, 0],
    X_train_2d[y_train == 0, 1],
    label="No",
    edgecolor="k"
)

plt.scatter(
    X_train_2d[y_train == 1, 0],
    X_train_2d[y_train == 1, 1],
    label="Yes",
    edgecolor="k"
)

plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("kNN Decision Boundary – Bank Marketing")
plt.legend()
plt.show()
```



In [ ]: